Fase 6 - CI/CD con GitHub Actions y Self Hosted Runner

Objetivo

Automatizar el despliegue de la aplicacion utilizando GitHub Actions, eliminando tareas manuales y permitiendo que cada push al repositorio desencadene la actualizacion automatica de la infraestructura y de la aplicacion.

Situacion inicial

Antes de esta fase, cada modificacion en Flask requeria conectarse manualmente al servidor Ubuntu mediante SSH y ejecutar docker compose up -d --build para reconstruir y desplegar los cambios.

Primer enfoque evaluado

Inicialmente se intento implementar despliegue remoto mediante SSH desde GitHub Actions utilizando el action appleboy/ssh-action y autenticacion basada en claves SSH.

Configuracion realizada

Se genero un par de claves SSH exclusivo para GitHub Actions. Se configuraron los Repository Secrets HOST, USERNAME y SSH_KEY dentro del repositorio GitHub.

Problema encontrado - Conexion SSH

Los workflows fallaban mostrando errores de timeout durante la conexion SSH. Luego del analisis se descubrio que GitHub Actions intentaba conectarse a la direccion privada 192.168.0.236 del laboratorio.

Analisis del problema

Se verifico que la maquina virtual estaba utilizando una direccion IP privada dentro de la red domestica. GitHub Actions se ejecuta desde Internet y no puede alcanzar direcciones privadas como 192.168.x.x. La solucion hubiese requerido IP publica y port forwarding en el router.

Decision arquitectonica

Se decidio abandonar el enfoque SSH tradicional y migrar hacia un Self Hosted Runner, una solucion mas segura y comun en laboratorios y entornos corporativos.

Implementacion de Self Hosted Runner

Se creo e instalo un GitHub Self Hosted Runner directamente sobre Ubuntu. El runner fue registrado contra el repositorio devops-lab y configurado para recibir trabajos directamente desde GitHub.

Workflow de despliegue

Se implemento un workflow deploy.yml utilizando runs-on: self-hosted, permitiendo ejecutar tareas directamente sobre la maquina virtual del laboratorio.

Problema encontrado - Runner Offline

Al publicar el workflow los jobs quedaron en estado Queued indefinidamente. El analisis demostro que el runner se encontraba Offline porque no estaba ejecutandose. Una vez iniciado mediante run.sh, los workflows comenzaron a ejecutarse.

Problema encontrado - Conflicto de contenedores

Durante la ejecucion del pipeline aparecio el error indicando que el contenedor flask-app ya existia. Docker no podia reutilizar el mismo nombre debido a configuraciones duplicadas.

Diagnostico realizado

Se inspeccionaron todos los archivos docker-compose.yml del laboratorio y se descubrio que existian dos despliegues distintos definiendo el mismo container_name flask-app.

Resolucion

Se identifico que app/docker-compose.yml pertenecia a una etapa anterior del proyecto. El archivo fue eliminado para unificar la administracion de la aplicacion bajo el docker-compose principal ubicado en la raiz del repositorio.

Validacion final

Se eliminaron recursos heredados, se ejecuto nuevamente el workflow y posteriormente se modifico app.py para comprobar que el despliegue automatico funcionaba. Tras realizar un nuevo git push, GitHub Actions reconstruyo y desplego correctamente la aplicacion.

Conceptos aprendidos

CI/CD, GitHub Actions, Self Hosted Runners, Workflows YAML, Repository Secrets, Automatizacion de despliegues, Integracion continua, Despliegue continuo y troubleshooting de pipelines.

Competencias adquiridas

GitHub Actions, Self Hosted Runner, Automatizacion de Docker Compose, Analisis de logs de workflows, Diagnostico de errores de red, Diagnostico de runners y resolucion de conflictos entre stacks Docker.

Resultado final

El laboratorio ahora cuenta con un pipeline funcional donde un git push desencadena automaticamente la reconstruccion y despliegue de la aplicacion, eliminando intervenciones manuales y acercando el entorno a practicas DevOps reales.